GOVT COLLEGE WOMEN UNIVERSITY OF FAISALABAD

Department of Computer Science

Assignment no 2

OF

Analysis of Algorithm

Submitted by: Tarteel Basharat

Roll no: 23263

Section: BSCS (Morning C) 6th

Submitted to: Mam Sadia Sehar

Insertion sort:

Insertion Sort sorts an array **like arranging cards in your hand**.
It picks one new element at a time and inserts it into the **correct position on the left side** (the sorted part).

Best Case (Already Sorted Array)

Example:

[1, 2, 3, 4, 5]

Since the array is already sorted, **no shifting is needed**.

Dry Run:

Pass 1:
Key = 2 → 2 ≥ 1 → no change
Array: [1, 2, 3, 4, 5]

Pass 2:
Key = 3 → 3 ≥ 2 → no change

Pass 3:
Key = 4 → no change

Pass 4:
Key = 5 → no change

Steps Count (Best Case)

- Comparisons = **n − 1**
- Shifts = **0**
- Time Complexity = **O(n)**

If n = 5
Comparisons = 4

Worst Case (Reverse Sorted Array)

Example:

[5, 4, 3, 2, 1]

Each new element must be moved to the **beginning**, so maximum shifting happens.

Dry Run:

Pass 1:
Key = 4
Compare with 5 → shift
Insert at start

Array: [4, 5, 3, 2, 1]
Steps = 1 shif

Pass 2:
Key = 3
Shift 5
Shift 4

Array: [3, 4, 5, 2, 1]
Steps = 2 shift

Pass 3:
Key = 2
Shift 5, 4, 3

Array: [2, 3, 4, 5, 1]
Steps = 3 shifts

Pass 4:
Key = 1
Shift 5, 4, 3, 2

Array: [1, 2, 3, 4, 5]
Steps = 4 shifts

Steps Count (Worst Case)

Total shifts =
1 + 2 + 3 + 4 = **10**

Worst Case Formula:

$n(n - 1) / 2$

For n = 5
$= 5 \times 4 / 2$
= **10**

Time Complexity = **$O(n^2)$**

Case	Condition	Time
Best	Already sorted	O(n)
Worst	Reverse sorted	$O(n^2)$

Bubble Sort:

Bubble Sort repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.
Big elements "bubble up" to the end — just like bubbles rise to the surface.**Best Case (Already Sorted Array)**

Example:

[1, 2, 3, 4, 5]

Array already sorted → **no swaps needed**

Dry Run:

Pass 1:
1 ≤ 2 → no swap
2 ≤ 3 → no swap
3 ≤ 4 → no swap
4 ≤ 5 → no swap

Array: [1, 2, 3, 4, 5]

Since no swap happened, algorithm stops

early.

Steps Count (Best Case)

- Comparisons = **n − 1**
- Swaps = **0**
- Time Complexity = **O(n)** (if optimized with swap check)

If n = 5
Comparisons = 4

Worst Case (Reverse Sorted Array)

Example:

[5, 4, 3, 2, 1]

Maximum swaps required.

Dry Run:

Pass 1:

5 & 4 → swap → [4, 5, 3, 2, 1]
5 & 3 → swap → [4, 3, 5, 2, 1]
5 & 2 → swap → [4, 3, 2, 5, 1]
5 & 1 → swap → [4, 3, 2, 1, 5]

Largest element reached end.

Swaps = 4

Pass 2:

4 & 3 → swap → [3, 4, 2, 1, 5]
4 & 2 → swap → [3, 2, 4, 1, 5]
4 & 1 → swap → [3, 2, 1, 4, 5]

Swaps = 3

Pass 3:

3 & 2 → swap → [2, 3, 1, 4, 5]
3 & 1 → swap → [2, 1, 3, 4, 5]

Swaps = 2

Pass 4:

2 & 1 → swap → [1, 2, 3, 4, 5]

Swaps = 1

Steps Count (Worst Case)

Total swaps =
4 + 3 + 2 + 1 = **10**

Worst Case Formula:

n(n – 1) / 2

For n = 5
= 5 × 4 / 2
= **10**

Time Complexity = **$O(n^2)$**

Case	Condition	Time
Best	Already sorted	O(n)
Worst	Reverse sorted	$O(n^2)$

Selection Sort:

Selection Sort **selects the smallest (or largest) element** from the unsorted part and places it at the correct position in the sorted part.
Think of it like **finding the minimum card** from your deck and putting it in order, one by one.

Best Case (Already Sorted Array)

Example:

[1, 2, 3, 4, 5]

Even if the array is already sorted, Selection Sort **still scans the remaining elements** to find the minimum.

Dry Run:

Pass 1:
Find min in [1, 2, 3, 4, 5] → min = 1 → already at correct place
Array: [1, 2, 3, 4, 5]

Pass 2:
Find min in [2, 3, 4, 5] → min = 2 → already at correct place

Pass 3:
Find min in [3, 4, 5] → min = 3 → already at correct place

Pass 4:
Find min in [4, 5] → min = 4 → already at correct place

Pass 5:
Only [5] remains → done

Steps Count (Best Case)

- Comparisons = **n(n − 1)/2** → always same
- Swaps = **0** (if optimized to avoid swapping with itself)
- Time Complexity = **O(n²)**

Even for sorted array, **Selection Sort does all comparisons**.

Worst Case (Reverse Sorted Array)

Example:

[5, 4, 3, 2, 1]

We need to find the minimum each time and swap it to the start.\

Dry Run:

Pass 1:
Find min in [5, 4, 3, 2, 1] → min = 1 → swap with 5
Array: [1, 4, 3, 2, 5]

Pass 2:
Find min in [4, 3, 2, 5] → min = 2 → swap with 4
Array: [1, 2, 3, 4, 5]

Pass 3:
Find min in [3, 4, 5] → min = 3 → already correct → no swap
Array: [1, 2, 3, 4, 5]

Pass 4:
Find min in [4, 5] → min = 4 → already correct → no swap

Pass 5:
Only [5] remains → done

Steps Count (Worst Case)

- Comparisons = **n(n − 1)/2** → 5×4/2 = 10
- Swaps = **n − 1** → 4
- Time Complexity = **O(n²)**

Case	Condition	Comparisons	Swaps	Time
Best	Already sorted	$O(n^2)$	0	$O(n^2)$
Worst	Reverse sorted	$O(n^2)$	$n - 1$	$O(n^2)$